

直接存储访问技术。

OmniSci (MapD) 是一个基于 GPU 和 CPU 混合架构的内存数据库, 包含内存数据库模式的 CPU 版本与加速模式的 GPU 版本。OmniSci 通过将用户查询编译为 CPU 和 GPU 上执行的机器码提高查询性能, 通过向量化查询执行和 GPU 代码优化技术提高查询执行性能。查询执行时, CPU 负责查询解析, 与 GPU 计算并行执行。

传统数据库的基础硬件假设正在发生变化, 非易失性内存成为大容量、低成本、高性能的新内存, 改变传统数据库面向易失性内存而设计的日志、缓存、恢复等机制; 硬件加速器成为高性能计算 (high performance computing, HPC) 的主流平台, 也将成为高性能数据库的计算平台, 面向 GPU 架构及 CPU-GPU 异构计算平台的数据库也成为一个新的技术发展趋势。

17.5 内存数据库前沿技术与展望

硬件技术的发展为内存数据库提供了硬件基础, 以 GPU、FPGA 为代表的异构计算体系结构的发展成为高性能计算平台的主流技术。

从存储技术的发展来看, 大内存已成为高性能计算的主流配置, 非易失性内存 NVRAM 已进入产品化阶段, 高性能多层堆叠 SSD 缩小了传统内存与外部存储之间的性能差距, RDMA、NVLink (GPU 架构) 等数据传输通信技术降低了网络访问延迟, 异构存储提供了基于性能、成本的不同组合方案。

传统的数据库基于同构硬件架构而设计。异构硬件架构需要数据库扩展对不同硬件的支持, 将数据库的存储引擎及查询引擎与异构硬件进行优化匹配, 为用户提供满足不同性能与成本需求的解决方案。

1. 面向千核处理器的内存数据库技术

随着 ARM 处理器成为 TOP 500 新型高性能计算平台, 基于 ARM 处理器的千核处理器计算平台成为新的高性能内存数据库硬件平台。与 X86 处理器相比, ARM 处理器的核心集成度更高, 基于多路处理器的千核计算平台为高性能内存数据库提供了更加强大的并行计算能力, 也需要数据库在千核并行处理技术上对传统的并发控制和查询处理进行优化, 以更好地发挥 ARM 处理器的并行计算能力。数据库传统的并发控制技术在千核处理器平台上不能自动适应, 更高的并发度甚至可能降低事务处理的整体性能, 需要根据千核处理器的硬件特征优化并行控制算法, 也需要对传统的并发控制算法进行深入评估, 确定最优并行度。另一方面, 千核处理器在分析处理负载中提供了更强大的并行计算能力, 但需要通过面向 NUMA 结构优化多核并行算法性能, 最大化千核并行计算性能。

2. 面向异构内存的内存数据库技术

NVRAM 的出现使内存成为持久存储设备, 补足了内存数据库在事务处理 ACID 特性中长期缺失的 D 特性, 从而使内存数据库成为真正意义上的完整数据库系统。NVRAM 具有内存的访问特性, 但其物理结构决定了它在内存访问延迟、带宽等性能上与 DRAM 存在差距, 在优

化访问方式上也有所差异,不能从当前的内存数据库架构直接升级,需要面向 DRAM-NVRAM 的异构内存特性而优化设计内存数据库存储和查询引擎,将较小的但数据访问频繁的数据或中间数据存储在 DRAM,而将较大的、数据访问延迟要求低的数据存储在 NVRAM,优化两种不同容量、不同访问性能、不同成本的混合内存上的综合查询性能。当前,在 NVRAM 数据库技术研究中,基于混合内存的存储优化、索引优化、查询优化等是主要的研究方向,主要的优化策略是尽量减少 NVRAM 在读、写、访问粒度上相对 DRAM 的较大延迟,通过 DRAM 优化 NVRAM 上的存储访问性能。

3. 面向 GPU-GPU 异构计算平台的内存数据库技术

GPU 与 FPGA 作为硬件加速器可以显著提升特定的计算性能,基于硬件加速器的异构计算平台为高性能数据库提供了硬件基础。其中 GPU 数据库发展最为成熟,已有一系列 GPU 数据库产品成为新一代高性能数据库的代表性技术。从数据库引擎实现技术来看, GPU 数据库有两条主要的技术路线。

(1) GPU 内存数据库

随着 GPU 技术的发展,更多核心、更大显存容量、更高数据通道性能(NVLink 或 PCIe 4.0/5.0)、多卡集成能力等特性,使 GPU 成为新的内存数据库平台,提供对一定规模数据集(基于数据压缩和多卡扩展存储容量)的高性能查询处理能力,从而获得远超过基于 CPU 和 DRAM 的内存数据库查询处理性能。

GPU 数据库是完全面向 GPU 特定的硬件架构而设计的数据库系统,通过实时编译的 cuda 代码生成技术和面向 GPU 存储结构的优化技术提高查询算子性能,在 TBps 级显存带宽性能的支持下提供支持百亿记录的毫秒级查询响应性能。

(2) GPU 加速引擎

GPU 加速引擎技术面向较大数据集的查询处理任务,通过将适合 GPU(而不适合 CPU)的计算密集型负载分配给 GPU 以达到提高查询算子性能的目标。当前, CPU 与 GPU 之间的 PCIe 通道带宽性能显著低于内存带宽和 GPU 显存带宽,成为 GPU 加速技术最主要的性能瓶颈。除数据压缩、传输优化、计算与数据传输流水并行等优化技术之外,提高 GPU 计算的数据局部性也是重要的优化方法,即从查询负载中分离出数据量小、计算量大、计算输出结果小的计算密集型负载并提高计算的局部性是一个有效的解决方案。从当前 GPU 数据库实现技术来看,面向关系操作算子的 GPU 加速技术因实时数据传输代价而降低了整体性能加速收益,因此定制适合 GPU 本地计算的操作算子是 GPU 加速技术的基础。

在 GPU 数据库的研究中,一方面需要根据 GPU 不同于 CPU 的硬件结构特点定制和优化 GPU 上的查询算法性能,另一方面需要根据 GPU 显存容量优化 CPU-GPU 混合存储模型,优化 CPU 与 GPU 间低速 PCIe 通道的数据传输代价。

4. HTAP 数据库引擎技术

传统的磁盘数据库受磁盘 I/O 带宽的影响而性能较低,需要将事务处理与执行时间较长的分析处理划分为独立的事务处理系统和分析处理系统,分别采用不同存储模型的查询优化方

法。内存数据库极大地提升了事务处理系统和分析处理系统的性能,尤其使分析处理系统不依赖于存储和计算成本较高的物化视图、索引等机制,支持实时分析处理。内存数据库性能的提升推动了混合事务分析处理技术的发展,即将事务处理系统与分析处理系统集成在一个数据库引擎内,实现实时的事务与分析处理。混合事务分析处理面对两种不同特征的负载处理,需要在存储模型和查询处理两方面实现融合与资源分配。

当前的混合事务分析处理技术主要体现在存储模型融合技术上,通常在数据库内部采用适合事务处理的行存储和适合分析处理的列存储混合引擎,通过一定的优化机制,实现内部行存储数据向列存储数据的迁移和存储结构转换。混合事务分析处理系统实现了事务处理引擎与分析处理引擎在数据库系统内部的集成,由数据库系统自动完成事务处理与分析处理任务以及行列存储转换,为用户提供统一的数据库平台。在分布式系统中,一种代表性的混合事务分析处理技术是:利用多个数据副本和日志更新策略保持事务处理副本与分析处理副本的一致性,通过不同副本的并行处理保证事务处理与分析处理负载同时具有较高的性能。

混合存储引擎技术解决了事务处理系统与分析处理系统在数据存储模型上的统一,在高负载应用场景下,分析处理负载需要占用大量处理器资源执行长程查询处理(long-term query),短事务处理需要与分析处理竞争处理器资源,降低了查询响应时间。异构计算平台为混合事务分析处理提供了新的解决方案,集成的高性能硬件加速器适用于分析处理负载执行,从而为异构计算平台上的混合事务分析处理系统释放处理器资源用于事务处理,使事务处理和分析处理负载在处理器资源上最大化隔离,减少两种负载对处理器资源的争用,提高两种负载各自的处理性能。当前 GPU 数据库通常作为独立的联机分析处理数据库引擎,主要处理数据仓库负载,未来将成为新型的混合事务分析处理计算平台,实现内存事务处理与 GPU 分析处理的集成与融合,为用户提供统一的数据库应用平台。

本章小结

大容量内存、非易失性内存、闪存、多核 CPU、众核处理器、高性能网络传输等硬件技术的发展,为内存数据库提供了良好的平台。虽然内存价格相对于传统的磁盘仍然很高,但内存数据库的软件结构相对更简单,代码执行效率更高,不需要复杂的索引、物化视图等传统的数据库调优技术,具有更好的性价比。

内存事务处理数据库的软件技术比较成熟,已大量应用于金融、电信等实时响应性能要求较高的应用领域,成为企业核心业务事务处理的解决方案。随着内存硬件价格的不断下降,内存分析处理数据库成为内存计算新的技术增长点,一些商业内存数据库近年来取得了巨大的市场份额,传统数据库厂商 Oracle、IBM 和 Microsoft 等分别推出了内存数据库产品及解决方案,新兴的内存数据库产品 SAP HANA 及 VectorWise 等也迅速被高端数据库应用所采纳。应用案例表明,即使是一些大型企业,其核心分析数据也能够完全存储于内存或内存数据库集群中以支持高性能分析处理。

当前的服务器已能够支持 TB 级内存,在数据压缩技术的支持下,内存数据库平台能够支持数倍甚至数十倍于物理内存的大数据处理任务,成为大数据应用的有效解决方案。随着新硬件技术的不断发展,内存数据库技术也将随之而不断发展。非易失性内存技术的成熟进一步为内存数据库技术的发展提供了硬件基础,推动内存数据库成为大数据实时分析处理的重要平台。新型处理器技术的发展推动高性能计算平台走向异构化,如何在异构计算平台上进一步提高内存数据库的性能和处理能力是未来的一个重要研究方向。

限于篇幅,本章许多技术仅作概要论述,感兴趣的读者可以阅读本章列出的参考文献。

习 题 17

1. 内存数据库和磁盘数据库有什么区别?
2. 磁盘数据库与内存数据库各适用于哪些应用场景?
3. 试述内存列存储模型的结构特点、在联机事务处理与联机分析处理应用场景中行存储结构和列存储结构的优缺点,分析存储模型与应用模式的关系。
4. 假设订单表 ORDERS 包含 16 个 INT 类型(4 B)的数据列,表中记录为 10 亿条,分别采用磁盘行存储结构和内存列存储结构。假设磁盘访问带宽为 100 MBps,内存带宽为 200 GBps。查询需要扫描表中的任意 3 个数据列,试估算在两种不同存储结构下的表扫描时间。
5. 试述内存数据库和硬件的相关性。多核 CPU、GPU、非易失性存储器等新硬件对内存数据库的设计有哪些影响,带来哪些性能收益?
6. 以内存哈希连接算法为例,试述在多核 CPU 平台和 GPU 平台如何根据硬件特性进行优化设计。

参考文献 17

- [1] 张延松,王珊.内存数据库技术与实现[M].北京:高等教育出版社,2016.
- [2] RAO J, ROSS K A. Making B+ trees cache conscious in main memory[C]. Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, 2000: 475-486.
- [3] LEE I H, SHIM J, LEE S G, et al. CST-trees: cache sensitive T-trees[C]. Proceeding of the 12th International Conference on Database Systems for Advanced Applications(DASFAA 2007), 2007: 398-409.
- [4] MANEGOLD S, BONCZ P, NES N, et al. Cache-conscious radix-decluster projections[C]. Proceedings of the 30th International Conference on Very Large Data Base, 2004, 30: 684-695.
- [5] BALKESSEN C, TEUBNER J, ALONSO G, et al. Main-memory hash joins on multi-core CPUs: Tuning to the underlying hardware[C]. Proceedings of the 29th IEEE International Conference on Data Engineering, 2013: 362-373.
- [6] KEMPER A, NEUMANN T, FINIS J, et al. Transaction processing in the hybrid OLTP&OLAP main-memory database system HyPer[J]. IEEE Data Engineering Bulletin, 2013, 36(2): 41-47.
- [7] SIKKA V, FÄRBER F, LEHNER W, et al. Efficient transaction processing in SAP HANA database: the end of a column store myth[C]. Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data,

2012; 731–742.

[8] HE B S, LU M, YANG K, et al. Relational query coprocessing on graphics processors[J]. ACM Transactions on Database Systems, 2009, 34(4): 1–39.

[9] HE B S, Yu J X. High-throughput transaction executions on graphics processors[C]. Proceedings of the VLDB Endowment, 2011, 4(5): 314–325.

[10] ZUKOWSKI M, BONCZ P A, NES N, et al. MonetDB/X100 – a DBMS in the CPU cache[J]. IEEE Data Engineering Bulletin, 2005, 28(2): 17–22.

[11] NEUMANN T, LEIS V, et al. Compiling database queries into machine code[J]. IEEE Data Engineering Bulletin, 2014, 37(1): 3–11.

[12] BALKESSEN C, ALONSO G, TEUBNER J, et al. Multi-core, main-memory joins: sort vs. hash revisited [C]. Proceedings of the VLDB Endowment, 2013, 7(1): 85–96.

[13] AREFYEVA I, BRONESKE D, CAMPERO G, et al. Memory management strategies in CPU/GPU database systems: a survey[C]. International Conference: Beyond Databases, Architectures and Structures (BDAS'18), 2018: 128–142.

[14] HALSTEAD R J, ABSALYAMOV I, NAJJAR W A, et al. FPGA-based multithreading for in-memory hash joins[C]. Proceedings of the 7th Biennial Conference on Innovative Data Systems Research (CIDR'15), 2015.

[15] CHENG X T, HE B, S LU M, et al. Efficient query processing on many-core architectures: a case study with Intel Xeon Phi processor[C]. Proceedings of the 2016 International Conference on Management of Data, 2016: 2081–2084.

[16] ABADI D, AILAMAKI A, ANDERSEN D, et al. The seattle report on database research[J]. ACM SIGMOD Record, 2019, 48(4): 44–53.

[17] SHANBHAG A, MADDEN S, YU X Y. A study of the fundamental performance characteristics of GPUs and CPUs for database analytics[C]. Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD'20), 2020: 1617–1632.

[18] YU X Y, YOUILL M, WOICIK M, et al. PushdownDB: accelerating a DBMS using S3 computation[C]. Proceedings of the 2020 IEEE 36th International Conference on Data Engineering (ICDE), 2020: 1802–1805.

[19] LU Y, YU X Y, CAO L, et al. Aria: a fast and practical deterministic OLTP database[C]. Proceedings of the VLDB Endowment, 2020, 13(12): 2047–2060.

[20] LU B T, HAO X P, WANG T Z, et al. Dash: scalable hashing on persistent memory[C]. Proceedings of VLDB Endowment. 2020, 13(8): 1147–1161.

[21] HAO X P, LERSCH L, WANG T Z, et al. PiBench online: interactive benchmarking of persistent memory indexes[C]. Proceedings of VLDB Endowment, 2020, 13(12): 2817–2820.

[22] HE Y J, LU J C, WANG T Z. CoroBase: coroutine-oriented main-memory database engine[Z]. 2020, CoRR abs/2010.15981.

[23] LIU J H, CHEN S M, WANG L J. LB+Trees: optimizing persistent index performance on 3DXPoint memory [C]. Proceedings of VLDB Endowment, 2020, 13(7): 1078–1090.

本章简要而系统地介绍区块链技术，主要包括区块链的概念与工作原理、区块链的发展进程、区块链系统的技术架构与关键技术、区块链与数据库的对比与融合等。

区块链是数据块的链接列表，是存储数据的一种新型数据结构。直观地看，它构成了数据更新的日志。

本章的目的不仅仅在于介绍区块链技术，而是力求从数据库视角出发，将区块链技术和数据库技术相结合，使其能够相互借鉴。在数据库系统中能够借鉴和应用区块链的去中心化技术、去信任理念和防篡改技术，使数据库应用更加安全可靠。同时，区块链能够从数据库查询优化和存储管理等技术中得到启发，进一步扩展其查询功能，提高事务处理效率，使得区块链的应用更加广泛深入。

18.1 区块链的概念与工作原理

区块链概念诞生至今仅十多年，目前已发展成为时下技术热点之一。区块链作为一种新型的分布式数据存储与处理系统，已成为数据库领域重要的研究和应用方向。

2016 年 12 月，区块链首次作为战略性前沿技术写入《“十三五”国家信息化规划》，我国将区块链定位为核心技术自主创新的重要突破口。

本章内容参考了中国信息通信研究院 (China Academy of Information and Communications Technology, CAICT, 简称中国信通院) 与可信区块链推进计划 (Trusted Blockchain Initiatives, TBI) 发布的若干版本白皮书^[1-3]，以及国内外相关文献与专家报告等。

区块链诞生时日毕竟尚短，技术成熟度有限。除比特币 (Bitcoin) 和以太坊 (Ethereum) 以外，区块链技术目前还缺乏大型应用实例的支撑，其标准体系尚未完备搭建，很多提法尚处于探讨研究阶段。本章内容和观点需要随着区块链技术和应用的不断发展进行修正和完善。

互联网是一场革命，它解决了信息的高效和廉价传输，但是它也带来了新问题，因为在网络上复制、传播、篡改一条信息的成本几乎为零，网络信息的可信度大打折扣。在货币领域，尽管电子货币带给人类极大便利，但目前的电子货币还必须依赖大量的第三方中介机构才能实